

Special Participation B: Coding Problem in HW 6 using Deepseek

Zachary's Karate Club

Yicheng Xiao

3042011144

jaredanjerry@berkeley.edu

? Question:

We will train a GNN to cluster people in the karate club in such that people who are more likely to associate with either the officer or Mr. Hi will be close together, while the distance between the 2 classes will be far. We will build a multi-layer Graph Convolutional Network (GCN) with the following layer-wise propagation rule:

$$H^{l+1} = \sigma\left(D^{-\frac{1}{2}} \tilde{A} D^{-\frac{1}{2}} H^l W^l\right) = \sigma\left(\tilde{A}^{\text{SymNorm}} H^l W^l\right)$$

where $\tilde{A}$ is the adjacency matrix that we discussed in Homework 4, except that now we add the identity to include self loops at every node for numerical stability. D is the degree matrix as defined in the past, H^l is the activation matrix of the l -th layer, and W is the weight matrix to be learned. σ is an activation function, in this case tanh.

Can you first tell me:

1. How to calculate A_{tilde} in python
2. How to get the adjacent matrix A_{hat} by implementing function `def get_adjacency_matrix(A_tilde):`
3. Generate the input feature matrix X , set it to be identity.

Think for 39 seconds

I'll help you with each part step by step.

1. How to calculate A_{tilde} in Python

```
import numpy as np
import scipy.sparse as sp

def calculate_A_tilde(A):
    """
    Calculate  $A_{\text{tilde}} = A + I$ , where  $A$  is the original adjacency matrix
    and  $I$  is the identity matrix (adding self-loops)

    Parameters:
    A: Original adjacency matrix (numpy array or scipy sparse matrix)

    Returns:
    A_tilde: Adjacency matrix with self-loops
    """
    n_nodes = A.shape[0]

    # Create identity matrix
    I = sp.identity(n_nodes)

    # Add self-loops:  $A_{\text{tilde}} = A + I$ 
    if sp.issparse(A):
        A_tilde = A + I
    else:
        A_tilde = A + np.eye(n_nodes)

    return A_tilde
```

2. How to get the symmetric normalized adjacency matrix A_{hat}

```
def get_adjacency_matrix(A_tilde):
    """
    Calculate the symmetric normalized adjacency matrix  $A_{\text{hat}} = D^{-1/2} A_{\text{tilde}} D^{-1/2}$ 

    Parameters:
    A_tilde: Adjacency matrix with self-loops
```

Returns:

A_hat: Symmetric normalized adjacency matrix

"""

Calculate degree matrix D

```
if sp.issparse(A_tilde):
    degrees = np.array(A_tilde.sum(axis=1)).flatten()
else:
    degrees = np.sum(A_tilde, axis=1)
```

Avoid division by zero for isolated nodes

```
degrees = np.maximum(degrees, 1.0)
```

Calculate $D^{-1/2}$

```
D_inv_sqrt = np.diag(1.0 / np.sqrt(degrees))
```

Calculate $A_{\text{hat}} = D^{-1/2} A_{\text{tilde}} D^{-1/2}$

```
if sp.issparse(A_tilde):
    # Convert to array for computation, or use sparse operations
    A_tilde_dense = A_tilde.toarray()
    A_hat = D_inv_sqrt @ A_tilde_dense @ D_inv_sqrt
else:
    A_hat = D_inv_sqrt @ A_tilde @ D_inv_sqrt
```

```
return A_hat
```

Alternative sparse implementation (more efficient for large graphs):

```
def get_adjacency_matrix_sparse(A_tilde):
    """
    Sparse implementation of symmetric normalization
    """
    # Calculate degree matrix
    degrees = np.array(A_tilde.sum(axis=1)).flatten()
    degrees = np.maximum(degrees, 1.0)

    # Calculate  $D^{-1/2}$  as a diagonal sparse matrix
    D_inv_sqrt = sp.diags(1.0 / np.sqrt(degrees))

    # Calculate  $A_{\text{hat}} = D^{-1/2} A_{\text{tilde}} D^{-1/2}$ 
    A_hat = D_inv_sqrt @ A_tilde @ D_inv_sqrt

    return A_hat
```

3. Generate the input feature matrix X as identity

```
def generate_feature_matrix(n_nodes):
    """
    Generate the input feature matrix X as identity matrix

    Parameters:
    n_nodes: Number of nodes in the graph

    Returns:
    X: Identity matrix of shape (n_nodes, n_nodes)
    """
    X = np.eye(n_nodes)
    return X
```

Complete Example for Karate Club

Example usage for Karate Club network

```
def setup_karate_club_data():
    """
    Setup the complete data for Karate Club GNN training
    """
    # Assuming you have the original adjacency matrix A for Karate Club
```

```

# This would typically come from your graph data
n_nodes = 34 # Karate club has 34 nodes

# Create a sample adjacency matrix (replace with actual Karate club data)
# For demonstration, creating a random symmetric adjacency matrix
A = np.random.randint(0, 2, (n_nodes, n_nodes))
A = np.tril(A) + np.tril(A, -1).T # Make symmetric
np.fill_diagonal(A, 0) # Remove self-loops initially

# 1. Calculate A_tilde
A_tilde = calculate_A_tilde(A)

# 2. Calculate A_hat
A_hat = get_adjacency_matrix(A_tilde)

# 3. Generate feature matrix X
X = generate_feature_matrix(n_nodes)

return A_tilde, A_hat, X

# Usage
A_tilde, A_hat, X = setup_karate_club_data()

print(f"A_tilde shape: {A_tilde.shape}")
print(f"A_hat shape: {A_hat.shape}")
print(f"X shape: {X.shape}")
print(f"X is identity matrix: {np.allclose(X, np.eye(X.shape[0]))}")

```

This setup gives you:

- **A_tilde**: Adjacency matrix with self-loops for numerical stability
- **A_hat**: Symmetrically normalized adjacency matrix ready for GCN layers
- **X**: Identity matrix as input features (each node has its own unique one-hot encoded feature)

The identity matrix as features means each node starts with a unique identifier, which is common when no other node features are available, as in the Karate club dataset.

🌟 Good Point!

The model gets all of them correct!

⚠️ Oops, something might be wrong...

Because there are no context prompted for the `get_adjacency_matrix` function `fill_diagonal` method is not used and need some adaptation to the current code framework.

? Question:

Then help me finish the following GNN Layer:

```
class GNN_Layer():
    """process a single GNN layer"""
    def __init__(self, input_dim, output_dim, name=''):
        self.name = name
        self.cache = {}
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.W = get_xavier_init(self.output_dim, self.input_dim)
        self.activation = np.tanh

    def __repr__(self):
        dims = (self.input_dim, self.output_dim)
        if self.name:
            return f"GNN_Layer: W{'_' + self.name}{dims}"
        else:
            return f"GNN_Layer: W{'_' + ''}{dims}"

    def forward_pass(self, A, X, W=None):
        """A here is the symmetrically normalized adjacency matrix
        and X is the input to the layer. We cached some values
        to use in the backward pass."""

        self.cache['A'] = A # (batch_size, batch_size)
        # TODO
        # self.cache['X'] = ?? # (input_node_feature_dim, batch_size)

        # if W is None:
        #     W = ??

        # H = ?? (hidden_dim, batch_size)
        # H = ?? (apply activation)
        # self.cache['H'] = ?? # (hidden_dim, batch_size)
        return # (batch_size, hidden_dim)

    def backward_pass(self, optimizer, need_update=True):
        # dtanh = ?? # (batch_size, output_dim)
        # optimizer.output contains the gradient from the next layer. It is multiplied by the batch
size,
        # so you'll need to divide by optimizer.batch_size to get the average gradient.
        d = np.multiply(optimizer.output, dtanh) # (batch_size, output_dim)

        # self.grad = ?? (batch_size, input_dim)

        optimizer.output = self.grad

        # dW = ?? # (output_dim, input_node_feature_dim)
        # dW_weight_decay = ??

        # if need_update: # Use the gradient descent update rule on W. Remember to include weight decay.
        #     self.W = ??

        return # (output_dim, input_node_feature_dim)
```

Think for 238 seconds

I'll help you complete the GNN layer implementation. Here's the full implementation:

```
class GNN_Layer():
    """process a single GNN layer"""
    def __init__(self, input_dim, output_dim, name=''):
        self.name = name
```

```

self.cache = {}
self.input_dim = input_dim
self.output_dim = output_dim
self.W = get_xavier_init(self.output_dim, self.input_dim)
self.activation = np.tanh

def __repr__(self):
    dims = (self.input_dim, self.output_dim)
    if self.name:
        return f"GNN_Layer: W{'_' + self.name}{dims}"
    else:
        return f"GNN_Layer: W{'_' + ''}{dims}"

def forward_pass(self, A, X, W=None):
    """A here is the symmetrically normalized adjacency matrix
    and X is the input to the layer. We cached some values
    to use in the backward pass."""

    self.cache['A'] = A # (batch_size, batch_size)
    self.cache['X'] = X # (input_node_feature_dim, batch_size)

    if W is None:
        W = self.W

    # H = A * X * W
    # A: (n_nodes, n_nodes), X: (n_nodes, input_dim), W: (input_dim, output_dim)
    H = A @ X @ W # (n_nodes, output_dim)
    H = self.activation(H) # Apply tanh activation
    self.cache['H'] = H # (n_nodes, output_dim)

    return H # (n_nodes, output_dim)

def backward_pass(self, optimizer, need_update=True):
    # Calculate derivative of tanh: d(tanh(x))/dx = 1 - tanh(x)^2
    H = self.cache['H']
    dtanh = 1 - H**2 # (n_nodes, output_dim)

    # optimizer.output contains the gradient from the next layer
    # Divide by batch_size to get average gradient
    dL_dH = optimizer.output / optimizer.batch_size # (n_nodes, output_dim)
    d = np.multiply(dL_dH, dtanh) # (n_nodes, output_dim)

    # Calculate gradients
    A = self.cache['A'] # (n_nodes, n_nodes)
    X = self.cache['X'] # (n_nodes, input_dim)

    # Gradient for previous layer: dL/dX = A^T @ d @ W^T
    self.grad = A.T @ d @ self.W.T # (n_nodes, input_dim)

    # Gradient for weights: dL/dW = X^T @ A^T @ d
    # But since A is symmetric (A = A^T), we can simplify
    dW = X.T @ A @ d # (input_dim, output_dim)

    # Add weight decay
    dW_weight_decay = dW + optimizer.weight_decay * self.W

    if need_update:
        # Update weights using gradient descent
        self.W = self.W - optimizer.learning_rate * dW_weight_decay

    # Set optimizer output for previous layer
    optimizer.output = self.grad

    return dW # (input_dim, output_dim)

```

Key points explained:

1. Forward Pass:

- $H = A @ X @ W$: Implements the GCN propagation rule
- Applies tanh activation function
- Caches values needed for backward pass

2. Backward Pass:

- **dtanh**: Derivative of tanh activation: $1 - \tanh(x)^2$
- **dL_dH**: Gradient from next layer, normalized by batch size
- **d**: Element-wise product of gradient and activation derivative
- **self.grad**: Gradient w.r.t input X for previous layer: $A^T @ d @ W^T$
- **dW**: Gradient w.r.t weights: $X^T @ A @ d$
- **Weight update**: Includes weight decay regularization

3. Dimensions:

- Input X: (n_nodes, input_dim)
- Output H: (n_nodes, output_dim)
- Weights W: (input_dim, output_dim)
- Adjacency A: (n_nodes, n_nodes)

Note: You'll also need the Xavier initialization function:

```
def get_xavier_init(fan_in, fan_out):
    """Xavier/Glorot initialization for weights"""
    scale = np.sqrt(2.0 / (fan_in + fan_out))
    return np.random.normal(0, scale, size=(fan_in, fan_out))
```

This implementation follows the standard GCN forward propagation and backpropagation through the graph convolution operation, properly handling the gradients for both the weights and the input features.

★ Good Point!

The general implementation is good. All the calculations are mathematically correct.

! Oops, something might be wrong...

1. However, it changes the dimension in the comment and will be inconsistent with the Softmax_Layer.
2. The weight decay is not scaled with the output_size, which may result in some problems.

? Question:

Can you please do not change the dimension given in the comment area after # but use transpose function to fulfill the requirement, because this has to align with another Softmax Layer then the dimension of the forward and backward need to stay consistent.

Think for 137 seconds

I'll fix the GNN layer to maintain the exact dimensions specified in the comments using transpose operations:

```
class GNN_Layer():
    """process a single GNN layer"""
    def __init__(self, input_dim, output_dim, name=''):
        self.name = name
        self.cache = {}
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.W = get_xavier_init(self.output_dim, self.input_dim)
        self.activation = np.tanh

    def __repr__(self):
        dims = (self.input_dim, self.output_dim)
        if self.name:
            return f"GNN_Layer: W{'_' + self.name}{dims}"
```

```

else:
    return f"GNN_Layer: W{'_' + ''}{dims}"

def forward_pass(self, A, X, W=None):
    """A here is the symmetrically normalized adjacency matrix
    and X is the input to the layer. We cached some values
    to use in the backward pass."""

    self.cache['A'] = A # (batch_size, batch_size)
    self.cache['X'] = X # (input_node_feature_dim, batch_size)

    if W is None:
        W = self.W

    # Transform dimensions to align with GCN operation
    # X is (input_node_feature_dim, batch_size) but we need (batch_size, input_node_feature_dim) for
matrix multiplication
    X_transposed = X.T # (batch_size, input_node_feature_dim)

    # H = A * X * W following the GCN propagation rule
    # A: (batch_size, batch_size), X_transposed: (batch_size, input_node_feature_dim), W:
(input_dim, output_dim)
    H_before_activation = A @ X_transposed @ W # (batch_size, output_dim)

    # Apply activation
    H = self.activation(H_before_activation) # (batch_size, output_dim)

    # Cache H as (hidden_dim, batch_size) - need to transpose
    self.cache['H'] = H.T # (hidden_dim, batch_size)

    return H # (batch_size, hidden_dim)

def backward_pass(self, optimizer, need_update=True):
    # Get cached values
    H_cached = self.cache['H'] # (hidden_dim, batch_size)
    A = self.cache['A'] # (batch_size, batch_size)
    X = self.cache['X'] # (input_node_feature_dim, batch_size)

    # Convert H back to (batch_size, hidden_dim) for calculations
    H = H_cached.T # (batch_size, hidden_dim)

    # Calculate derivative of tanh: d(tanh(x))/dx = 1 - tanh(x)^2
    dtanh = 1 - H**2 # (batch_size, output_dim)

    # optimizer.output contains the gradient from the next layer
    # Divide by batch_size to get average gradient
    dL_dH = optimizer.output / optimizer.batch_size # (batch_size, output_dim)
    d = np.multiply(dL_dH, dtanh) # (batch_size, output_dim)

    # Calculate gradients
    # X is (input_node_feature_dim, batch_size) - need to transpose for matrix multiplication
    X_transposed = X.T # (batch_size, input_node_feature_dim)

    # Gradient for previous layer: dL/dX = A^T @ d @ W^T
    grad_before_transpose = A.T @ d @ self.W.T # (batch_size, input_dim)
    self.grad = grad_before_transpose # (batch_size, input_dim)

    # Gradient for weights: dL/dW = X^T @ A^T @ d
    # But we need to work with the transposed dimensions
    dW = X @ A @ d # (input_node_feature_dim, output_dim)

    # Add weight decay
    dW_weight_decay = dW + optimizer.weight_decay * self.W

    if need_update:

```

```

        # Update weights using gradient descent
        self.W = self.W - optimizer.learning_rate * dW_weight_decay

    # Set optimizer output for previous layer (already in correct dimension: batch_size, input_dim)
    optimizer.output = self.grad

    return dW # (output_dim, input_node_feature_dim)

```

Key changes to maintain exact dimensions:

1. Forward Pass:

- Input X: (input_node_feature_dim, batch_size) → transpose to (batch_size, input_node_feature_dim) for matrix multiplication
- Output H: (batch_size, hidden_dim) as required
- Cache H: transpose to (hidden_dim, batch_size) as specified

2. Backward Pass:

- Recover H from cache: transpose from (hidden_dim, batch_size) back to (batch_size, hidden_dim)
- Input X: (input_node_feature_dim, batch_size) → transpose to (batch_size, input_node_feature_dim) for matrix operations
- self.grad: Maintains (batch_size, input_dim) as specified
- dW: Returns (output_dim, input_node_feature_dim) as specified

3. Matrix Operations:

- All operations now respect the dimension requirements in comments
- Used transpose operations to convert between row-major and column-major formats as needed
- Maintained consistency with the expected Softmax layer dimensions

This implementation ensures that the dimensions flow correctly through the network and will align properly with your subsequent Softmax layer.

★Good Point!

This is a followup question of the above question and Deepseek successfully handles the dimension problems!

? Question:

Then, please based on the former knowledge, fill in the following function:

```

# TODO
# gnn_layer_output = gnn_layer.forward_pass(A=??,
#                                           X=??)

# optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=labels)

```

Think for 67 seconds

Based on the previous implementation, here's how to fill in the function calls:

```

# TODO
# gnn_layer_output = gnn_layer.forward_pass(A=??,
#                                           X=??)

# optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=labels)

# Assuming:
# - A_hat is the symmetrically normalized adjacency matrix from get_adjacency_matrix()
# - X is the input feature matrix (identity matrix) from generate_feature_matrix()
# - gnn_layer is an instance of GNN_Layer
# - sm_layer is an instance of SoftmaxLayer
# - labels are the ground truth labels for the nodes

gnn_layer_output = gnn_layer.forward_pass(A=A_hat,
                                          X=X)

```

```
optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=labels)
```

Explanation:

1. GNN Layer Forward Pass:

- $A=A_{\text{hat}}$: Use the symmetrically normalized adjacency matrix A_{hat} that we computed earlier using `get_adjacency_matrix(A_tilde)`
- $X=X$: Use the input feature matrix X (identity matrix) that we generated using `generate_feature_matrix(n_nodes)`

2. Softmax Layer and Optimization:

- `sm_layer.forward_pass(X=gnn_layer_output)`: Pass the GNN layer output to the softmax layer for classification
- `optim(y_pred=..., y_true=labels)`: Compute the loss and gradients using the optimizer

Complete context with dimension flow:

```
# Setup data
A_tilde = calculate_A_tilde(A) # Adjacency matrix with self-loops
A_hat = get_adjacency_matrix(A_tilde) # Symmetrically normalized adjacency matrix
X = generate_feature_matrix(n_nodes) # Identity matrix as features

# Create GNN layer (example dimensions)
gnn_layer = GNN_Layer(input_dim=n_nodes, output_dim=hidden_dim)

# Forward pass through GNN
gnn_layer_output = gnn_layer.forward_pass(A=A_hat, X=X)
# gnn_layer_output shape: (batch_size, hidden_dim) as required

# Forward pass through softmax and optimization
optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=labels)
```

Dimension consistency:

- A_{hat} : (batch_size, batch_size) - matches the GNN layer expectation
- X : (input_node_feature_dim, batch_size) - matches the GNN layer expectation
- gnn_layer_output : (batch_size, hidden_dim) - will match the softmax layer's expected input dimension
- The softmax layer will then produce outputs that can be compared with `labels` for optimization

This maintains the exact dimension requirements specified in all the comments and ensures proper alignment between the GNN layer and subsequent softmax layer.

🌟 Good Point!

Correct!

? Question:

Now, use the GNN and Softmax layers implemented above to set up our GNN Network.

```
class GNN():
    """This class leverages the GNN layer implemented above by cascading them into a layer."""
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dim,
                  num_layers,):

        self.input_dim = input_dim
        self.output_dim = output_dim
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.layers = []
        first_gnn_layer = GNN_Layer(input_dim=??,
                                     output_dim=??,
                                     name='layer_0')

        self.layers.append(first_gnn_layer)

        for layer in range(num_layers):
            gnn_temp = GNN_Layer(input_dim=??,
                                  output_dim=??,
                                  name=f'layer_{layer}')
            self.layers.append(gnn_temp)

        last_gnn_layer = Softmax_Layer(input_dim=??,
                                         output_dim=??,
                                         name='sm_layer')
        self.layers.append(last_gnn_layer)

    def __repr__(self):
        return '\n'.join([str(layer) for layer in self.layers])

    def embedding(self, A, X):
        H = X
        for layer in self.layers[:-1]:
            # H = ??
            pass
        # return

    def forward_pass(self, A, X):
        # H = ??
        out = self.layers[-1].forward_pass(H)
        pass
        # return
```

And please complete the model initialization part:

```
# TODO
# gnn_model = GNN(
#     input_dim=??,
#     output_dim=??,
#     num_layers=2,
#     hidden_dim=[16, 2],
# )
# gnn_model
```

Here's the complete implementation of the GNN class using the specified layer structure:

```
class GNN():
    """This class leverages the GNN layer implemented above by cascading them into a layer."""
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dim,
                  num_layers,):

        self.input_dim = input_dim
        self.output_dim = output_dim
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.layers = []

        # First GNN layer: input_dim -> first hidden dimension
        first_gnn_layer = GNN_Layer(input_dim=input_dim,
                                     output_dim=hidden_dim[0],
                                     name='layer_0')
        self.layers.append(first_gnn_layer)

        # Hidden GNN layers
        for layer in range(1, num_layers):
            gnn_temp = GNN_Layer(input_dim=hidden_dim[layer-1],
                                  output_dim=hidden_dim[layer],
                                  name=f'layer_{layer}')
            self.layers.append(gnn_temp)

        # Final Softmax layer: last hidden dimension -> output_dim
        last_gnn_layer = Softmax_Layer(input_dim=hidden_dim[-1],
                                        output_dim=output_dim,
                                        name='sm_layer')
        self.layers.append(last_gnn_layer)

    def __repr__(self):
        return '\n'.join([str(layer) for layer in self.layers])

    def embedding(self, A, X):
        """Get node embeddings (output of the last GNN layer before softmax)"""
        H = X
        for layer in self.layers[:-1]: # All layers except the softmax layer
            H = layer.forward_pass(A, H)
        return H # Returns embeddings before softmax

    def forward_pass(self, A, X):
        """Full forward pass through all layers including softmax"""
        H = self.embedding(A, X) # Get embeddings
        out = self.layers[-1].forward_pass(H) # Pass through softmax layer
        return out

# TODO: Initialize the GNN model for Karate Club
gnn_model = GNN(
    input_dim=34, # 34 nodes in Karate club, identity matrix features
    output_dim=2, # 2 classes: Mr. Hi vs Officer
    num_layers=2,
    hidden_dim=[16, 2], # First hidden layer: 16 dim, second: 2 dim
)
gnn_model
```

Explanation:

1. Initialization:

- **First GNN layer:** Takes input_dim=34 (identity matrix features for 34 nodes) and outputs hidden_dim[0]=16

- **Hidden layers:** Each takes the previous layer's output as input and produces the next hidden dimension
- **Softmax layer:** Takes the last hidden dimension (`hidden_dim[-1]=2`) and produces `output_dim=2` (the two classes)

2. **Embedding method:**

- Processes input through all GNN layers but stops before the softmax layer
- Returns the node embeddings that can be used for clustering or visualization

3. **Forward pass method:**

- Calls `embedding()` to get the node representations
- Passes the embeddings through the softmax layer for classification
- Returns the final class probabilities

For the Karate Club specific setup:

- `input_dim=34`: Since we're using identity matrix features (each node has a 34-dimensional one-hot vector)
- `output_dim=2`: Binary classification - associating with Mr. Hi vs the Officer
- `hidden_dim=[16, 2]`:
 - First hidden layer reduces from 34 to 16 dimensions
 - Second hidden layer reduces from 16 to 2 dimensions (perfect for 2D visualization)
- The final softmax layer takes the 2D embeddings and produces 2-class probabilities

This architecture will learn to cluster the Karate club members such that nodes more likely to associate with Mr. Hi vs the Officer will have similar embeddings, while keeping the two classes separated in the embedding space.

🌟 **Good Point!**

In general it is correct.

⚠️ **Oops, something might be wrong...**

However, there are no such function as `self.embedding` exist.

? Question:

Complete the training loop function below. Note that the train loss and test loss are computed over a given set of nodes that is defined by our training and testing set.

```
def train(model,
          A_hat,
          X,
          y_true,
          nodes_to_be_used_for_training,
          nodes_to_be_used_for_testing,
          threshold_value=.5,
          ealy_stopping_steps=60,
          num_epochs=20000):
    """trains our gnn model"""

    accs = []
    training_losses = []
    testing_losses = []
    embeddings = []
    y_preds = []

    minimum_loss = 1e7
    ealy_stopping_counter = 0

    for epoch in range(num_epochs):
        # TODO
        # y_pred = ??
        training_optim(y_pred, y_true, nodes_to_be_used_for_training)

        for layer in reversed(model.layers):
            layer.backward_pass(training_optim, need_update=True)

        embeddings.append(model.embedding(A_hat, X))
        y_preds.append(y_pred)
        acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))[
            [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]
        ]
        accs.append(np.mean(acc_temp))

        # loss_temp = ??
        # train_loss_temp = ??
        # test_loss_temp = ??

        training_losses.append(train_loss_temp)
        testing_losses.append(test_loss_temp)

        if test_loss_temp < minimum_loss:
            minimum_loss = test_loss_temp
            ealy_stopping_counter = 0
        else:
            ealy_stopping_counter += 1

        if ealy_stopping_counter > ealy_stopping_steps:
            print("Training Stopped due to Early stopping!")
            break

        if epoch % 100 == 0:
            print(f"epoch #: {epoch+1} | train Loss: {train_loss_temp:.3f} | test Loss: {test_loss_temp:.3f}")

    training_losses = np.array(training_losses)
    testing_losses = np.array(testing_losses)
    y_preds = threshold(np.array(y_preds), threshold_value)
    return training_losses, testing_losses, accs, embeddings, y_preds
```

Here's the complete training loop function:

```
def train(model,
          A_hat,
          X,
          y_true,
          nodes_to_be_used_for_training,
          nodes_to_be_used_for_testing,
          threshold_value=.5,
          ealy_stopping_steps=60,
          num_epochs=20000):
    """trains our gnn model"""

    accs = []
    training_losses = []
    testing_losses = []
    embeddings = []
    y_preds = []

    minimum_loss = 1e7
    ealy_stopping_counter = 0

    for epoch in range(num_epochs):
        # Forward pass
        y_pred = model.forward_pass(A_hat, X)

        # Compute training loss and gradients
        training_optim(y_pred, y_true, nodes_to_be_used_for_training)

        # Backward pass through all layers
        for layer in reversed(model.layers):
            layer.backward_pass(training_optim, need_update=True)

        # Store embeddings and predictions
        embeddings.append(model.embedding(A_hat, X))
        y_preds.append(y_pred)

        # Calculate accuracy on test nodes
        acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))[
            [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]
        ]
        accs.append(np.mean(acc_temp))

        # Calculate losses
        loss_temp = training_optim.loss
        train_loss_temp = training_optim.compute_loss(y_pred, y_true, nodes_to_be_used_for_training)
        test_loss_temp = training_optim.compute_loss(y_pred, y_true, nodes_to_be_used_for_testing)

        training_losses.append(train_loss_temp)
        testing_losses.append(test_loss_temp)

        # Early stopping check
        if test_loss_temp < minimum_loss:
            minimum_loss = test_loss_temp
            ealy_stopping_counter = 0
        else:
            ealy_stopping_counter += 1

    if ealy_stopping_counter > ealy_stopping_steps:
        print("Training Stopped due to Early stopping!")
        break
```

```

    if epoch % 100 == 0:
        print(f"epoch #: {epoch+1} | train Loss: {train_loss_temp:.3f} | test Loss: {test_loss_temp:.3f}")

    training_losses = np.array(training_losses)
    testing_losses = np.array(testing_losses)
    y_preds = threshold(np.array(y_preds), threshold_value)
    return training_losses, testing_losses, accs, embeddings, y_preds

```

Key components explained:

1. Forward Pass:

```
y_pred = model.forward_pass(A_hat, X)
```

- Gets predictions for all nodes using the current model parameters

2. Optimization Step:

```
training_optim(y_pred, y_true, nodes_to_be_used_for_training)
```

- Computes loss and gradients using only training nodes
- Updates optimizer state

3. Backward Pass:

```

for layer in reversed(model.layers):
    layer.backward_pass(training_optim, need_update=True)

```

- Propagates gradients backward through all layers
- Updates weights in each layer

4. Loss Calculation:

- `loss_temp = training_optim.loss`: Gets the current loss value from optimizer
- `train_loss_temp`: Computes loss specifically on training nodes
- `test_loss_temp`: Computes loss specifically on testing nodes

5. Accuracy Calculation:

```

acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))[
    [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]
]

```

- Compares predicted vs true labels
- Only considers test nodes (nodes not in training set)
- Uses `argmax` to convert probabilities to class labels

Note: This implementation assumes that:

- `training_optim` is an optimizer object that has:
 - A callable interface `optim(y_pred, y_true, nodes)`
 - A `loss` attribute storing the current loss
 - A `compute_loss(y_pred, y_true, nodes)` method
- `y_true` and `y_pred` are one-hot encoded
- The `threshold` function converts probabilities to binary predictions

The training loop properly handles the separation between training and test nodes, implements early stopping, and tracks multiple metrics for monitoring training progress.

! Oops, something might be wrong...

The implementation seems to be correct, but because lack of context, it fails to use the existing function and need further adjustment.

? Question:

Explain why we obtain a 100% on accuracy on our test set, yet we see in the plot above that 2 samples seem to be misclassified.

The discrepancy between the reported 100% test accuracy and the visualization showing 2 misclassified samples can be explained by several factors:

1. Accuracy Calculation vs Visualization Timing

```
# Accuracy is calculated during training
acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))[
    [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]
]
```

The accuracy is computed **during training** on the **test nodes only**, but the visualization likely shows:

- The **final model predictions** after training completes
- Potentially on **all nodes** (not just test nodes)
- Using a **different threshold** than the argmax used in accuracy calculation

2. Thresholding Differences

```
# Accuracy uses argmax (hard classification)
np.argmax(y_pred, axis=1) # Always picks the highest probability class

# But final predictions use thresholding
y_preds = threshold(np.array(y_preds), threshold_value) # Uses threshold_value=0.5
```

The issue: During training, accuracy is calculated using argmax, but the plot might use the thresholded predictions where:

- Some predictions might be very close to 0.5
- With threshold=0.5, these borderline cases might flip classes
- But with argmax, they still get classified “correctly”

3. Training vs Final Model State

The 100% accuracy might be from:

- **Early stopping point** where test loss was minimum
- But the **final model** after all epochs might have slightly different weights
- The visualization uses the **final trained model**, not the early stopping checkpoint

4. Node Set Mismatch

```
# Accuracy uses only test nodes
[i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]

# But plot might show ALL nodes
```

The plot likely visualizes **all nodes**, including:

- Training nodes that were misclassified
- Nodes that were borderline cases

5. Probabilistic vs Hard Classification

```
# During training accuracy calculation
pred_class = np.argmax(y_pred, axis=1) # Hard classification

# But the model outputs probabilities
# A prediction of [0.51, 0.49] is "correct" with argmax
# But visually, it's very close to the decision boundary
```

Solution to Investigate:

Add debugging to see the actual probabilities:

```
# Add this to understand the discrepancy
final_y_pred = model.forward_pass(A_hat, X)
test_nodes_mask = [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]

print("Test node predictions (probabilities):")
for i in test_nodes_mask:
    prob = final_y_pred[i]
    true_class = np.argmax(y_true[i])
```

```
pred_class = np.argmax(prob)
print(f"Node {i}: true={true_class}, pred={pred_class}, probs={prob}")

# Check if any test nodes have probabilities near 0.5
borderline_nodes = [i for i in test_nodes_mask if np.max(final_y_pred[i]) < 0.6]
print(f"Borderline test nodes: {borderline_nodes}")
```

The most likely explanation is that the **visualization uses thresholded predictions on all nodes**, while the **reported accuracy uses argmax on test nodes only**, and there might be some borderline cases that appear misclassified in the plot but were considered correct during training accuracy calculation.